

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Paradigmas de Programación (CE1106)
Primer Semestre 2026

Documento de Atributos

Tarea 4: spaCEinvaders

Profesor:

[Luis Alonso Barboza Artavia](#)

Estudiantes:

- Frander Picado Mena 2022101904
- Samuel Morales Cascante 2024130487
- Dylan Elizondo Picado 2023086320

Fecha de Entrega:

11 de junio de 2026

Contenido

Atributo Diseño (DI)	3
1. Requerimientos del Sistema	3
1.1 Requerimientos funcionales	3
1.2 Requerimientos no funcionales	4
1.3 Observación de diseño	5
2. Alternativas de Solución	5
3. Valoración de las Opciones de Solución	7
4. Selección de la Propuesta Final	9
5. Diseño de la Alternativa Seleccionada	10
5.1 Diagrama de bloques general	10
5.2 Diagrama de clases UML del servidor	11
5.3 Máquina de estados del controlador	12
5.4 Flujo general del servidor	12
5.5 Descripción de algoritmos	14
6. Validación del Diseño	14
7. Conclusión de validación	15
Atributo Aprendizaje Continuo (AC)	16
2.1. Identifica sus necesidades de aprendizaje	16
2.2. Identifica tecnologías nuevas y emergentes	16
2.3. Implementa acciones o estrategias	17
2.4. Evalúa de forma crítica la eficacia de las estrategias	18
2.5 Evaluación crítica	18
Referencias	19

Atributo Diseño (DI)

1. Requerimientos del Sistema

A partir de la especificación del proyecto *spaCEinvaders* y de los criterios del documento de atributos, se identifican los requerimientos funcionales y no funcionales que debe satisfacer la solución propuesta. El sistema debe mantener la lógica central del juego en un servidor Java, operar con clientes implementados en C, comunicarse mediante sockets, soportar al menos dos clientes jugadores y un cliente espectador, y contar con un controlador físico de dos botones para el manejo del cañón. Además, la solución debe respetar el uso de programación imperativa en C, programación orientada a objetos en Java, uso de listas como estructura de datos y al menos dos patrones de diseño en el servidor.

1.1 Requerimientos funcionales

RF-01. El servidor deberá implementar toda la lógica del juego.

Esto incluye la administración del estado global de la partida, la creación y destrucción de entidades y el control del flujo general del juego.

RF-02. El servidor deberá llevar la puntuación del jugador.

La puntuación deberá actualizarse según el tipo de enemigo eliminado y los eventos del juego.

RF-03. El servidor deberá administrar las vidas del jugador.

La partida iniciará con tres vidas y cada impacto recibido reducirá una vida. Al eliminarse todos los extraterrestres, el jugador recuperará una vida y el juego reiniciará con mayor velocidad de desplazamiento enemiga.

RF-04. El servidor deberá crear y destruir extraterrestres durante la partida.

Deberá contemplar los distintos tipos de enemigos descritos en la especificación y sus valores de puntuación correspondientes.

RF-05. El servidor deberá generar OVNI's con valores de puntos configurables.

El valor deberá poder variar según el mensaje enviado durante la ejecución.

RF-06. El servidor deberá gestionar la velocidad de desplazamiento de los extraterrestres.

La velocidad deberá poder incrementarse dinámicamente a medida que avanza la partida.

RF-07. El servidor deberá administrar el estado de los bunkers.

El sistema deberá representar su porcentaje de salud y reflejar su deterioro progresivo durante el juego.

RF-08. El sistema deberá soportar al menos dos clientes jugadores simultáneos. Esto es un requisito explícito de revisión y debe contemplarse desde el diseño de red.

RF-09. El sistema deberá permitir un cliente espectador. Este cliente podrá unirse a una partida existente, pero únicamente en modo de observación.

RF-10. La comunicación entre servidor y clientes deberá realizarse mediante sockets.

La arquitectura de red debe basarse en esta tecnología, sin sustituirla por mecanismos alternativos.

RF-11. El controlador físico deberá permitir movimiento del cañón mediante dos botones.

Un toque deberá interpretarse como movimiento a la izquierda y dos toques como movimiento a la derecha.

RF-12. El controlador físico deberá incluir un botón de disparo.

El envío de la acción de disparo deberá integrarse con la comunicación hacia el sistema principal.

1.2 Requerimientos no funcionales

RNF-01. El servidor deberá desarrollarse en Java.

RNF-02. Los clientes deberán desarrollarse en C.

RNF-03. El controlador físico deberá programarse en C.

Para efectos del presente documento, y con la autorización indicada por el equipo, se tomará como plataforma un ESP32.

RNF-04. El código en C deberá seguir el paradigma imperativo.

Esto incluye el uso de structs, manejo manual de memoria y constantes separadas en archivos dedicados.

RNF-05. El código en Java deberá seguir el paradigma orientado a objetos.

El sistema deberá organizarse mediante clases, paquetes y relaciones de diseño coherentes.

RNF-06. El servidor deberá implementar al menos dos patrones de diseño formales.

La especificación sugiere alternativas como Observer, Abstract Factory, Adapter o Factory.

RNF-07. El sistema deberá utilizar listas como estructura de datos para manejar entidades dinámicas.

No se deben sustituir por estructuras estáticas que limiten el comportamiento esperado del nodo.

RNF-08. El sistema deberá incluir documentación suficiente y coherente con el tamaño del proyecto.

La evaluación valora la calidad de la documentación tanto como la funcionalidad.

RNF-09. El proyecto deberá entregar ejecutables funcionales para revisión.

La especificación exige ejecutables tanto en C como en Java.

RNF-10. El diseño deberá contemplar la posibilidad de validar el cumplimiento de los requisitos.

Esto es necesario para poder justificar la solución final con una lista de verificación.

1.3 Observación de diseño

Aunque la especificación original indica Raspberry Pi Pico para el control físico, en este proyecto se asumirá ESP32 como plataforma de control, dado que el equipo ya lo tomó como decisión técnica válida. Esta adaptación no altera los objetivos del sistema, siempre que se mantengan el lenguaje C, la comunicación serial y el comportamiento funcional requerido.

2. Alternativas de Solución

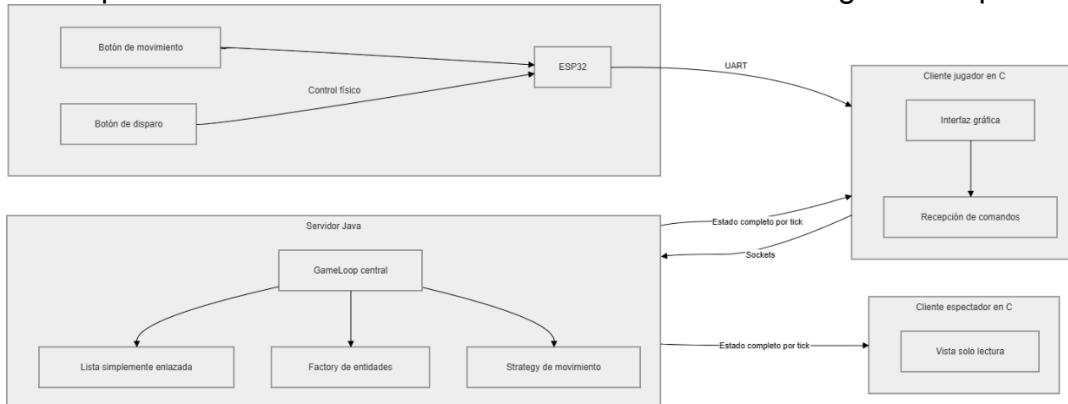
Para el desarrollo de *spaCEinvaders* se consideraron dos alternativas de solución viables, ambas compatibles con la especificación del proyecto y suficientemente robustas como para no ser descartadas de inmediato. La comparación se centró en la forma de sincronizar el estado del juego, la estructura de datos para manejar entidades dinámicas, la organización interna del servidor y el impacto de la solución en complejidad, rendimiento, mantenibilidad y uso de recursos. De acuerdo con el documento de atributos, cada alternativa debe incluir diagrama y análisis objetivo, por lo que en esta sección se presentan ambas propuestas con su respectiva justificación técnica.

Alternativa 1: Sincronización completa del estado, lista simplemente enlazada y servidor centralizado

Esta alternativa plantea una arquitectura de juego **centrada en el servidor**, donde el servidor Java mantiene toda la lógica principal, actualiza el estado global de la partida en un ciclo fijo y transmite a los clientes un resumen completo del estado del juego en cada iteración. Los clientes en C funcionan como interfaces gráficas de recepción y renderizado, mientras que el controlador físico envía comandos de entrada al cliente jugador mediante UART.

En esta propuesta, las entidades dinámicas del juego, como enemigos y proyectiles, se gestionan mediante **listas simplemente enlazadas**. Esta elección reduce el costo de memoria por nodo y simplifica la implementación inicial, aunque hace más costosa la eliminación de elementos en medio de la lista, porque para suprimir un nodo normalmente se requiere recorrer la estructura desde el inicio hasta encontrar el nodo anterior.

Desde el punto de vista del diseño de software, esta alternativa puede apoyarse en patrones como **Factory** para crear enemigos y oleadas, y **Strategy** para variar el comportamiento de movimiento de los extraterrestres según la etapa del juego.



Análisis técnico de la Alternativa 1

Esta alternativa es atractiva por su simplicidad conceptual. El cliente jugador recibe el estado completo del juego y no necesita reconstruir eventos ni mantener una lógica local compleja. Además, al utilizar listas simplemente enlazadas, el consumo de memoria por entidad es menor.

Sin embargo, su principal debilidad está en el tráfico de red y en el costo de mantenimiento del estado. Al enviar todo el tablero con frecuencia, el servidor aumenta el volumen de datos transmitidos, lo que puede volverse menos eficiente conforme crece el número de enemigos o se incrementa la cantidad de actualizaciones por segundo. Asimismo, las operaciones de eliminación en listas simples son menos convenientes cuando el sistema debe borrar entidades dinámicamente con mucha frecuencia.

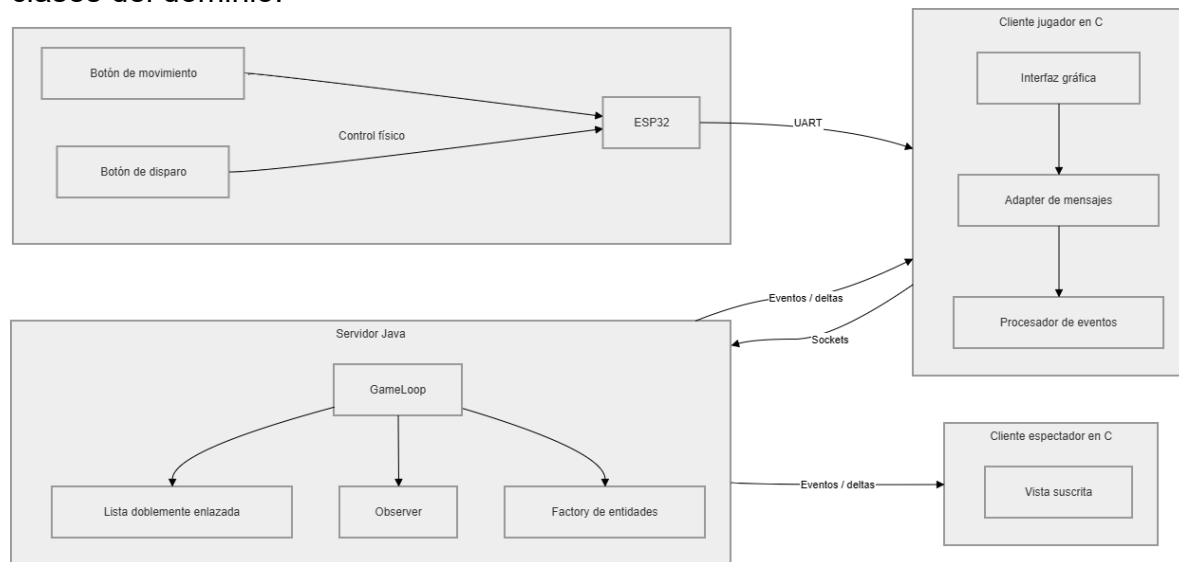
Alternativa 2: Sincronización por eventos, lista doblemente enlazada y arquitectura orientada a eventos

Esta alternativa propone una arquitectura más refinada, donde el servidor Java sigue siendo el dueño absoluto de la lógica del juego, pero en lugar de reenviar siempre el estado completo, transmite únicamente **eventos o cambios puntuales**: creación de enemigos, destrucción de entidades, variación de vidas, modificación de velocidad o cambios en los bunkers. Los clientes C reconstruyen visualmente el estado a partir de esos eventos.

Las entidades del juego se administran mediante listas doblemente enlazadas, lo que facilita la navegación hacia adelante y hacia atrás, y mejora la manipulación

de nodos cuando se necesita retirar un enemigo, un proyectil o cualquier otra entidad dinámica. Aunque cada nodo consume más memoria que en una lista simple, la estructura ofrece mayor flexibilidad en un contexto donde las eliminaciones y recorridos son frecuentes.

En el servidor, esta alternativa encaja naturalmente con patrones como Observer, para distribuir cambios a varios clientes, y Adapter, para traducir los mensajes recibidos o generados por el sistema a formatos internos manejables por las clases del dominio.



Análisis técnico de la Alternativa 2

Esta alternativa es más eficiente cuando el sistema debe manejar numerosas actualizaciones pequeñas, ya que reduce el volumen de datos transmitidos por red. También mejora la modularidad del servidor, porque cada evento puede tratarse como una unidad independiente de cambio. En términos de estructura de datos, la lista doblemente enlazada ofrece mayor control sobre la inserción y eliminación de nodos.

Su desventaja principal es una mayor complejidad de implementación. El sistema debe mantener consistencia entre eventos, estado interno y representación en pantalla, lo que incrementa el esfuerzo de depuración. Además, el cliente necesita lógica adicional para interpretar los cambios y reconstruir correctamente el estado visible, por lo que la solución demanda más disciplina de integración.

3. Valoración de las Opciones de Solución

Para valorar las alternativas se consideraron criterios técnicos y de contexto que están explícitamente alineados con el documento de atributos: complejidad, rendimiento, uso de recursos, impacto social/ambiental y aplicabilidad al problema. La comparación no busca descartar una opción de forma simplista, sino mostrar por qué cada una aporta ventajas reales bajo condiciones distintas.

3.1 Criterios de valoración

Se utilizaron los siguientes criterios:

- **Complejidad de implementación:** esfuerzo inicial requerido para construir e integrar la solución.
- **Rendimiento de red:** cantidad de información transmitida entre servidor y clientes.
- **Consumo de memoria:** impacto de la estructura de datos en el uso interno de recursos.
- **Mantenibilidad y escalabilidad:** facilidad para extender el sistema sin reescrituras profundas.
- **Uso de recursos externos:** necesidad de hardware o componentes adicionales.
- **Impacto ambiental:** relación entre la solución elegida y la reducción de residuos o elementos intermedios.

3.2 Matriz de valoración cualitativa

La siguiente tabla usa una escala de 1 a 5, donde 1 representa el desempeño menos favorable en ese criterio y 5 el más favorable.

Criterio	Alternativa 1	Alternativa 2	Comentario
Complejidad de implementación	5	3	La Alternativa 1 es más simple de construir y depurar.
Rendimiento de red	2	5	La Alternativa 2 reduce el tráfico al transmitir solo eventos.
Consumo de memoria	5	3	La lista simple requiere menos memoria por nodo.
Mantenibilidad y escalabilidad	3	5	La arquitectura por eventos es más flexible para crecer.
Uso de recursos externos	5	2	La Alternativa 1 evita hardware adicional para I2C.
Impacto ambiental	5	2	La menor cantidad de componentes intermedios favorece la sostenibilidad.

3.3 Interpretación de la valoración

La **Alternativa 1** resulta más favorable cuando se prioriza la simplicidad, el uso reducido de memoria y la facilidad de implementación inicial. Esto la convierte en una opción atractiva para un prototipo funcional que deba integrarse rápido y con menos puntos de falla.

La **Alternativa 2** destaca en eficiencia de red, mantenibilidad y escalabilidad. Al transmitir eventos en lugar de enviar el estado completo del juego, reduce la carga de comunicaciones y permite una arquitectura más limpia a largo plazo. Sin embargo, exige mayor complejidad de integración y un cliente más inteligente para interpretar correctamente los cambios.

3.4 Conclusión de la valoración

En términos generales, ambas alternativas son viables, pero responden a prioridades distintas. La primera favorece la rapidez de implementación y la sencillez estructural; la segunda ofrece una solución más robusta y escalable, con mejor comportamiento en red. Por ello, la decisión final debe basarse en el equilibrio entre facilidad de desarrollo, eficiencia técnica y capacidad de evolución del sistema.

4. Selección de la Propuesta Final

A partir de la valoración realizada en la sección anterior, se selecciona como propuesta final una solución que combina los elementos más ventajosos de las alternativas analizadas. En particular, se adopta la lógica de la **Alternativa 2**, basada en sincronización por eventos, uso de listas doblemente enlazadas y una arquitectura orientada a eventos para el servidor; mientras que el control físico se mantiene bajo el esquema común definido en ambas alternativas, es decir, comunicación serial mediante **UART** entre el ESP32 y el sistema principal. Esta decisión no responde a una preferencia arbitraria, sino a la necesidad de equilibrar eficiencia, coherencia estructural y viabilidad técnica dentro de las restricciones del proyecto.

La principal razón para seleccionar la Alternativa 2 en el núcleo del sistema es que el servidor debe administrar toda la lógica del juego, incluyendo la puntuación, las vidas, la creación y destrucción de enemigos, el estado de los bunkers, la velocidad de desplazamiento y la interacción con múltiples clientes. Bajo estas condiciones, un enfoque basado en eventos resulta más adecuado que el envío del estado completo del juego en cada ciclo, porque reduce la cantidad de información transmitida y mejora la organización interna del sistema. En lugar de reenviar continuamente toda la escena, el servidor notifica únicamente los cambios relevantes, lo que facilita una comunicación más limpia y escalable.

La selección de listas doblemente enlazadas también se justifica por la naturaleza dinámica del juego. Durante la partida se crean y destruyen constantemente entidades como alienígenas, balas y otros elementos temporales. En este

contexto, una lista doblemente enlazada ofrece una manipulación más flexible de nodos, ya que permite ajustar con mayor facilidad las referencias al insertar o eliminar elementos. Aunque utiliza más memoria que una lista simple, su comportamiento es más conveniente para un sistema con actualizaciones continuas y colisiones frecuentes.

En cuanto al control físico, se mantiene la comunicación por UART porque ambas alternativas la contemplan como mecanismo de enlace entre el ESP32 y la computadora. Por tanto, este aspecto no define la selección entre una alternativa u otra, sino que constituye una condición compartida que ya resulta adecuada para la implementación. UART continúa siendo la opción más práctica para este proyecto porque no exige hardware intermedio adicional y permite una integración directa con el cliente jugador.

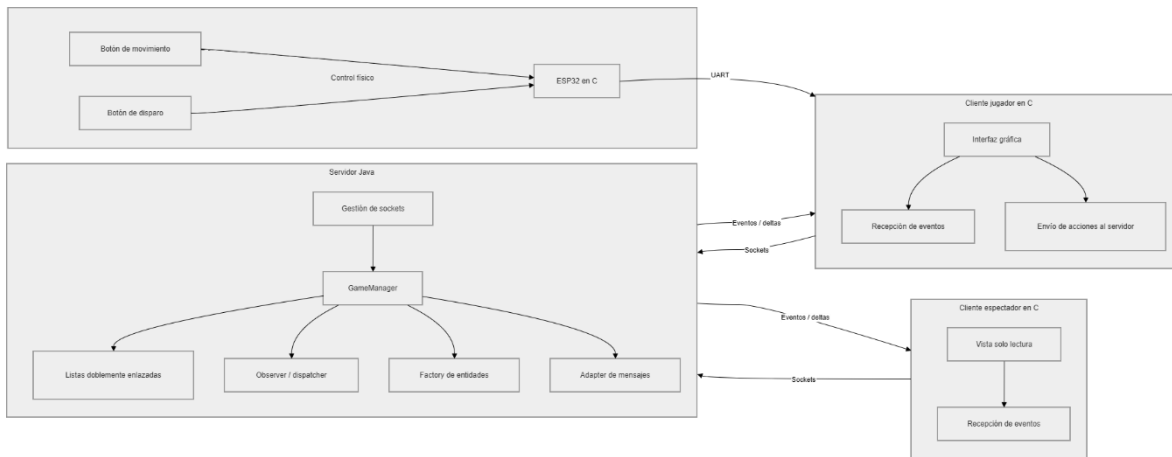
Desde el punto de vista de diseño de software, la propuesta final también favorece el uso de patrones de diseño formales en el servidor. La arquitectura seleccionada se adapta de forma natural al patrón **Observer**, que permite distribuir cambios a varios clientes conectados; al patrón **Factory**, que facilita la creación de distintos tipos de entidades del juego. Esta combinación fortalece la organización del servidor y contribuye a una solución más mantenible.

En consecuencia, la propuesta final queda definida de la siguiente manera: el **servidor en Java** administrará toda la lógica del juego mediante una arquitectura basada en eventos; los **clientes en C** actuarán como interfaces de juego y observación; la comunicación entre servidor y clientes se realizará mediante **sockets**; el control físico se implementará en **ESP32 programado en C**; y las entidades dinámicas se gestionarán mediante **listas doblemente enlazadas**. Con ello se obtiene una solución coherente con la especificación, técnicamente sólida y más adecuada para el crecimiento futuro del sistema.

5. Diseño de la Alternativa Seleccionada

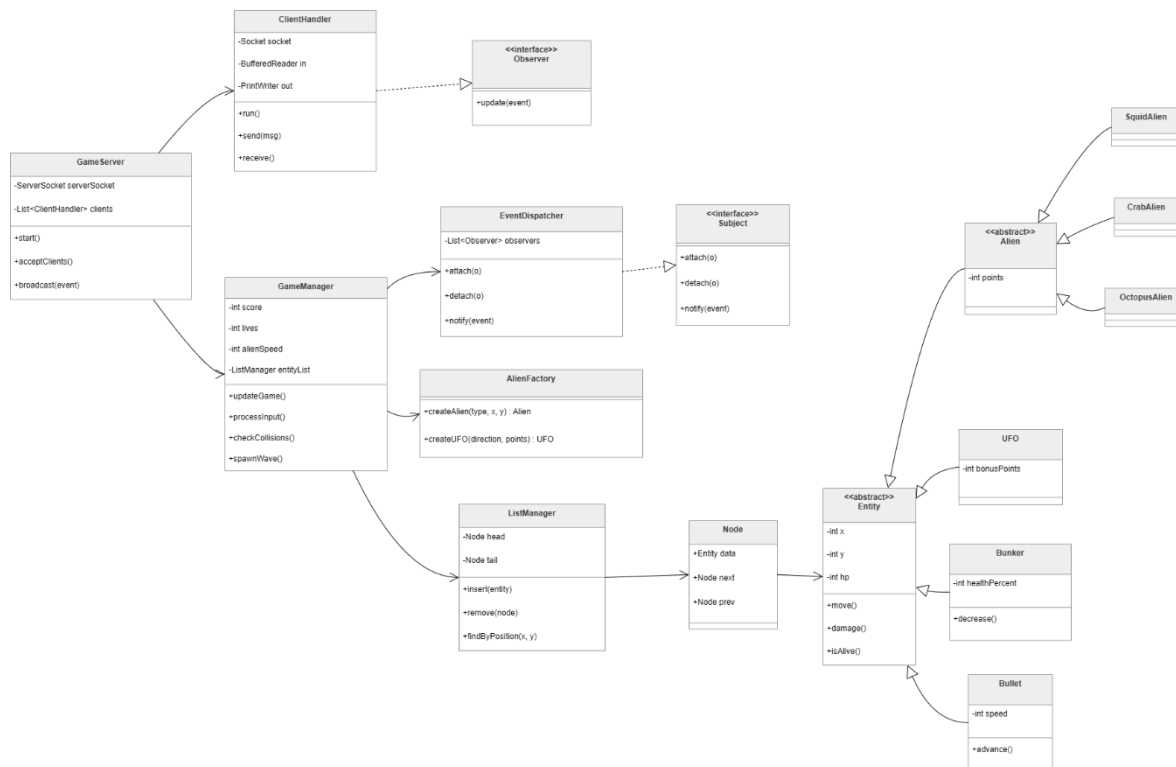
5.1 Diagrama de bloques general

El diseño general se organiza alrededor de un servidor Java que administra el estado completo del juego. Desde ese servidor se conectan los clientes en C, tanto jugadores como espectador. El control físico ESP32 se conecta a la computadora mediante UART y envía comandos de movimiento y disparo al cliente jugador. El servidor notifica únicamente eventos relevantes, mientras que los clientes reproducen visualmente el estado recibido.



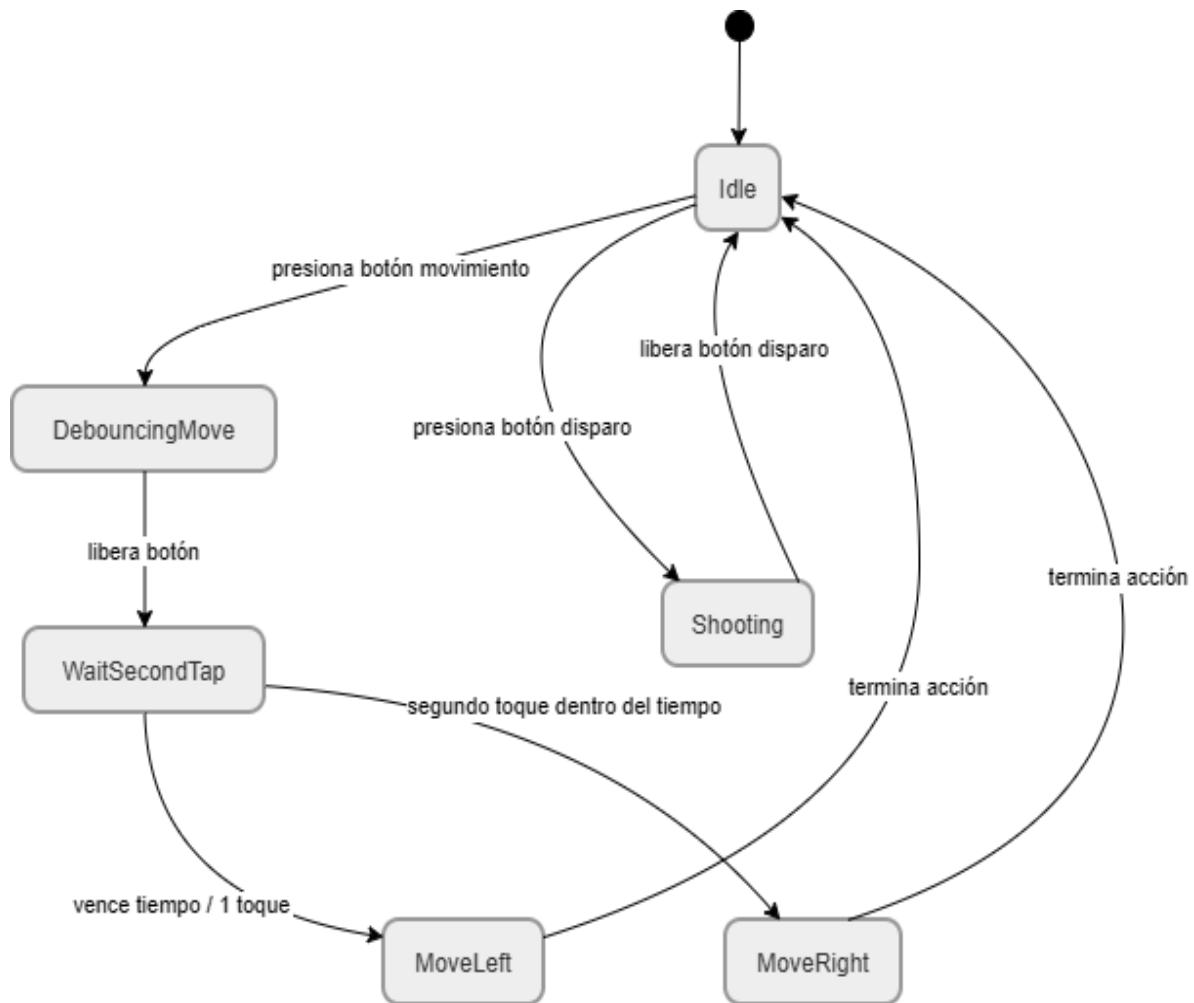
5.2 Diagrama de clases UML del servidor

El servidor se diseña con una estructura orientada a objetos que separa responsabilidades: manejo de red, lógica del juego, generación de entidades, administración de listas y difusión de eventos. Esto facilita mantener el código modular y aplicar patrones de diseño formalmente.



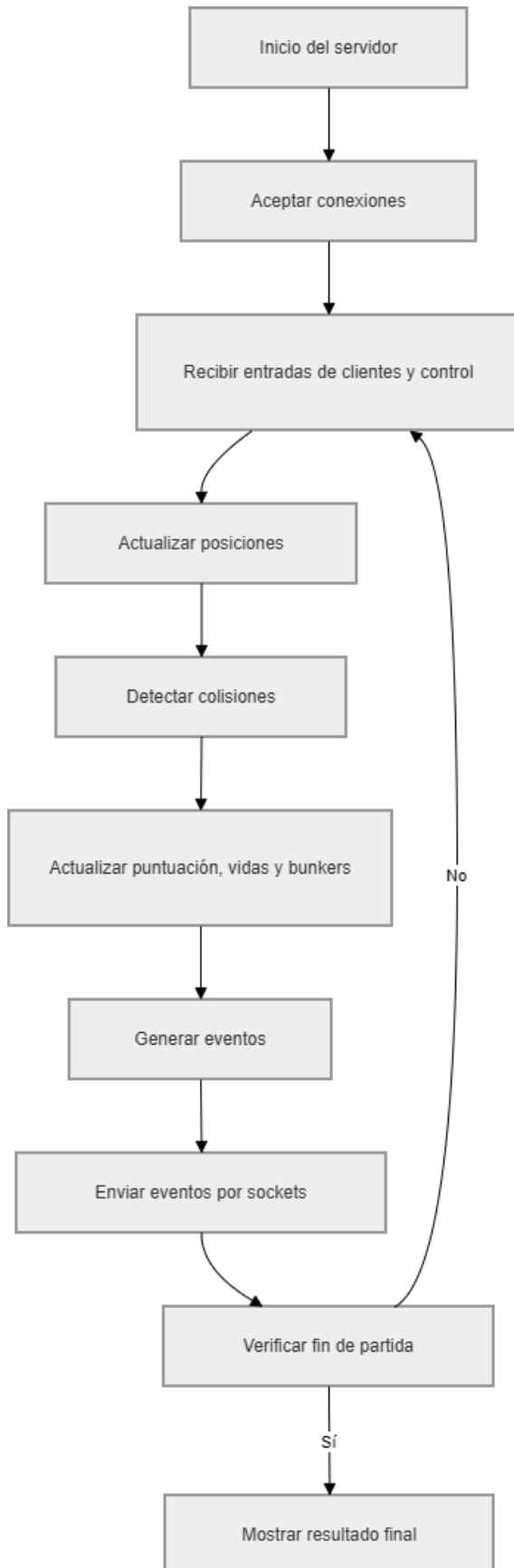
5.3 Máquina de estados del controlador

El controlador físico debe interpretar un botón para movimiento y otro para disparo. El botón de movimiento distingue entre toque simple y doble toque dentro de una ventana de tiempo definida; el botón de disparo envía la acción correspondiente mientras esté activo.



5.4 Flujo general del servidor

El servidor ejecuta un ciclo continuo de actualización. Recibe entradas, actualiza posiciones, detecta colisiones, modifica el estado de la partida y notifica a los clientes los eventos relevantes.



5.5 Descripción de algoritmos

Algoritmo 1. Interpretación del controlador físico

El ESP32 lee ambos botones mediante interrupciones o sondeo con antirrebote. Si el botón de movimiento detecta un toque simple, se genera el comando de movimiento a la izquierda. Si detecta dos pulsaciones dentro del tiempo permitido, se genera el comando de movimiento a la derecha. Si se presiona el botón de disparo, se transmite el evento correspondiente al cliente jugador, quien lo envía al servidor por socket.

Algoritmo 2. Inserción y eliminación de entidades

Cada entidad del juego se almacena como un nodo de lista doblemente enlazada. Al crear una nueva entidad, el nodo se inserta al final o en la posición requerida. Cuando una colisión elimina una entidad, el servidor localiza el nodo correspondiente y reajusta los punteros prev y next para retirarlo sin romper la estructura. Después se libera memoria para evitar fugas.

Algoritmo 3. Sincronización del estado del juego

En cada ciclo del juego, el servidor evalúa la posición de enemigos, balas, bunkers y jugadores. Si ocurre un cambio, como la destrucción de un alien o la pérdida de una vida, se genera un evento y se notifica a los clientes conectados. Así, el cliente no necesita resolver la lógica del juego, solo presentar la información recibida.

6. Validación del Diseño

La validación del diseño se realiza comparando los requerimientos del proyecto con los componentes definidos en la propuesta final. La solución seleccionada cumple con los criterios exigidos por la especificación: servidor en Java, clientes en C, sockets, control físico, estructuras de datos dinámicas y patrones de diseño en el servidor.

Requerimiento	Evidencia de diseño	Estado
Servidor en Java	La lógica central del juego, el gestor de eventos y el manejo de sockets se implementan en Java.	Cumple
Clientes en C	El cliente jugador y el cliente espectador se implementan en C.	Cumple
Lógica total en el servidor	GameManager centraliza puntuación, vidas, colisiones, velocidad y generación de entidades.	Cumple
Tres tipos de enemigos con puntaje distinto	Se modelan con clases separadas de alienígena y atributo de puntos.	Cumple
OVNI con puntos configurables	La fábrica de entidades permite crear OVNI con valor variable.	Cumple
Cuatro bunkers con salud variable	Bunker maneja porcentaje de salud y reducción progresiva.	Cumple
Dos clientes	GameServer mantiene lista de clientes y soporta	Cumple

jugadores mínimo	múltiples conexiones.	
Cliente espectador	Se conecta al servidor en modo solo lectura.	Cumple
Sockets obligatorios	La comunicación cliente-servidor se realiza mediante sockets TCP.	Cumple
Control físico de dos botones	El ESP32 interpreta movimiento y disparo con dos botones.	Cumple
Un toque / dos toques	La máquina de estados del controlador distingue ambos casos.	Cumple
Comunicación del control	El ESP32 se comunica con la PC mediante UART.	Cumple
Programación imperativa en C	El control y los clientes se desarrollan con structs y manejo manual de memoria.	Cumple
Programación OO en Java	El servidor se estructura con clases, jerarquías y encapsulación.	Cumple
Al menos dos patrones de diseño	Se integran Observer y Factory en el servidor.	Cumple
Uso de listas	Las entidades dinámicas se administran mediante listas doblemente enlazadas.	Cumple
Lista de verificación para validación	Esta tabla verifica el cumplimiento punto por punto.	Cumple

7. Conclusión de validación

El diseño propuesto satisface los requerimientos esenciales del proyecto y del documento de atributos. La arquitectura final es coherente con la especificación, permite justificar la elección técnica realizada y deja claramente documentados los componentes que soportan la funcionalidad esperada. Además, la selección de UART para el control físico y de listas doblemente enlazadas para las entidades dinámicas ofrece un balance razonable entre simplicidad, eficiencia y mantenibilidad.

Atributo Aprendizaje Continuo (AC)

El atributo de Aprendizaje Continuo evalúa la capacidad del equipo para reconocer sus necesidades de aprendizaje, adaptarse a tecnologías nuevas y emergentes, aprender de manera autónoma y ejercer pensamiento crítico frente al cambio tecnológico. Para este proyecto, dichos aspectos resultan especialmente relevantes debido a la integración de componentes heterogéneos: un servidor en Java, clientes en C, un controlador físico basado en ESP32 y comunicación mediante sockets y UART.

2.1. Identifica sus necesidades de aprendizaje

Aunque el equipo contaba con bases previas en programación imperativa, manejo de memoria dinámica y uso de estructuras de datos, el desarrollo de *spaCEinvaders* exigió fortalecer varias competencias específicas. La necesidad principal estuvo en la integración de sistemas heterogéneos, es decir, en lograr que un cliente gráfico en C, un servidor concurrente en Java y un microcontrolador actuando como controlador físico pudieran intercambiar información de forma estable y en tiempo real. Esta integración no es trivial, porque cada componente utiliza paradigmas, herramientas y restricciones distintas.

Otra necesidad importante fue el dominio de sockets TCP y concurrencia en Java. El servidor debía atender varios clientes simultáneamente sin bloquear la ejecución principal del juego, por lo que fue necesario comprender la gestión de hilos, la comunicación bidireccional y la sincronización de mensajes. Esta necesidad surgió directamente de los requerimientos del proyecto, que exigen soporte para al menos dos clientes jugadores y un cliente espectador.

También se identificó la necesidad de profundizar en el uso de ESP32 como controlador físico programado en C. Aunque el manejo básico de esta placa era conocido, el reto consistió en adaptarla para interactuar como periférico de entrada dentro de una arquitectura de red, manteniendo una comunicación serial estable y una detección confiable de botones mediante temporización y antirrebote.

Finalmente, se reconoció una necesidad de aprendizaje en organización técnica del trabajo, depuración conjunta y documentación de ingeniería. Dado que el proyecto combina varias capas de software y hardware, no basta con que cada módulo funcione por separado: también debe integrarse correctamente, documentarse con claridad y validarse contra los requisitos del sistema.

2.2. Identifica tecnologías nuevas y emergentes

Durante el desarrollo del proyecto se identificaron varias tecnologías y enfoques que contribuyeron al aprendizaje continuo del equipo. La primera fue la integración de microcontroladores ESP32 en arquitecturas de red. En lugar de funcionar como un dispositivo aislado, el ESP32 se utilizó como parte activa del sistema, enviando comandos al cliente jugador mediante comunicación serial. Esta adaptación permitió ampliar el uso del microcontrolador hacia un contexto más cercano a sistemas distribuidos y embebidos.

La segunda tecnología considerada fue el uso de inteligencia artificial generativa como apoyo consultivo para revisar documentación técnica y acelerar la comprensión de ciertos conceptos. Su función no fue reemplazar el criterio del equipo, sino servir como herramienta de consulta para validar ideas, revisar alternativas y organizar la redacción técnica.

La tercera tecnología emergente fue la diagramación como código, especialmente mediante Mermaid. Este enfoque permitió producir diagramas de bloques, UML y máquinas de estados de manera textual, lo que facilita la edición, la trazabilidad y la inclusión directa en herramientas como diagrams.net. En un proyecto con múltiples componentes y relaciones entre clases, esta práctica mejora la precisión de la documentación técnica.

Finalmente, se identificó como aprendizaje relevante el uso de entornos de desarrollo multitecnología, ya que el proyecto obligó a trabajar simultáneamente con Java, C y programación embebida en ESP32 dentro de un mismo flujo de integración. Esta convivencia de herramientas y lenguajes fortaleció la capacidad del equipo para adaptarse a diferentes contextos de desarrollo.

2.3. Implementa acciones o estrategias

Para atender las necesidades identificadas, el equipo aplicó varias estrategias de aprendizaje y ejecución. La primera fue la asignación dinámica de tareas, donde los integrantes asumieron responsabilidades según la parte del sistema que les correspondía o dominaban mejor. Esta estrategia permitió avanzar en paralelo con el cliente, el servidor y el controlador, aunque también hizo evidente la necesidad de una coordinación más estricta en ciertos momentos del proyecto.

La segunda estrategia consistió en el desarrollo modular con pruebas aisladas. Antes de completar la integración total, se probaron componentes de forma independiente mediante datos simulados o *mock data*, lo que permitió depurar la lógica interna sin depender todavía del comportamiento de otros módulos. Esta forma de trabajo fue útil para avanzar sin bloquear el progreso del proyecto completo.

La tercera estrategia fue la realización de sesiones intensivas de integración conjunta. Cuando el acoplamiento entre módulos comenzó a generar incompatibilidades, el equipo detuvo temporalmente el desarrollo aislado y priorizó la depuración colaborativa. Esto facilitó estabilizar la comunicación entre cliente, servidor y controlador, y permitió resolver errores que no se detectaban en pruebas unitarias.

Como apoyo adicional, el equipo recurrió a la documentación técnica, a la revisión de requisitos del proyecto y a la diagramación textual para mantener una visión clara del diseño final. Esto ayudó a conservar coherencia entre lo que se planeaba, lo que se implementaba y lo que se documentaba.

2.4. Evalúa de forma crítica la eficacia de las estrategias

La eficacia de las estrategias implementadas se evaluó mediante una comparación entre el nivel inicial de dominio y el nivel final alcanzado en áreas clave del proyecto. Se empleó una escala de 1 a 10, donde 1 representa un dominio muy bajo y 10 un dominio muy alto. Los resultados se resumen a continuación:

Área evaluada	Nivel inicial	Nivel final	Logro final	Interpretación
Implementación de sockets multilenguaje	3	8	80%	Mejora alta; se logró una comunicación estable entre Java y C.
Manejo de concurrencia	5	8	80%	Mejora alta; se fortaleció la comprensión del servidor concurrente.
Adaptación de hardware (ESP32)	6	9	90%	Mejora muy alta; se integró con éxito el controlador físico.
Planificación y sincronización del equipo	4	5	50%	Mejora moderada; persistieron desfases en tiempos de integración.
Resolución de conflictos de integración	5	8	80%	Mejora alta; la depuración conjunta fue eficaz.

A partir de esta tabla, el promedio de logro en las áreas estrictamente técnicas de software y hardware fue de **83.3%**, lo que respalda que las estrategias aplicadas fueron efectivas en el aprendizaje técnico del proyecto. En contraste, el componente de planificación y sincronización del equipo alcanzó un **50%**, mostrando que esa sigue siendo el área con mayor oportunidad de mejora.

2.5 Evaluación crítica

La evidencia cuantitativa muestra que las estrategias de aprendizaje fueron especialmente efectivas en lo técnico, sobre todo en sockets, concurrencia e integración con ESP32. En estas áreas, el equipo logró consolidar competencias nuevas y resolver problemas que al inicio del proyecto representaban una barrera importante. Por otro lado, la coordinación temporal entre integrantes no alcanzó el mismo nivel de efectividad, lo que sugiere que el aprendizaje más débil no fue de programación, sino de gestión colaborativa y planificación.

En conclusión, el proyecto permitió evidenciar aprendizaje continuo en tres niveles: técnico, tecnológico y organizativo. El avance más sólido se observó en la integración de sistemas heterogéneos y en el dominio de herramientas

emergentes; la principal área de mejora quedó en la sincronización del equipo y la administración del tiempo. Esta combinación de resultados demuestra una evolución real en la capacidad de aprender, adaptarse y responder a un entorno tecnológico cambiante.

Referencias

Escuela de Ingeniería en Computadores. (2026). *Tarea 4: spaCEinvaders*. Instituto Tecnológico de Costa Rica.

Escuela de Ingeniería en Computadores. (2026). *Documento de atributos: Diseño en Ingeniería y Aprendizaje Continuo*. Instituto Tecnológico de Costa Rica.